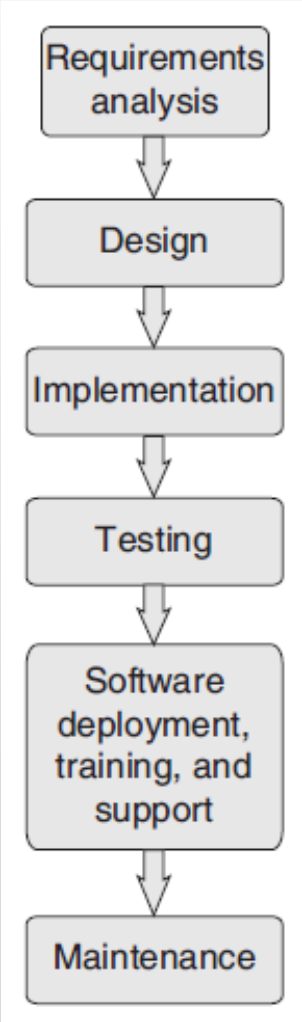
Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents

Problem Solving Strategies



Problem Solving Strategies

- **In requirements analysis, user's expectations are gathered to know why the software has to be built.**

- **In the design phase, a plan of actions is made before the actual development process can start.**

Problem Solving Strategies

- **In implementation phase, the designed algorithms are converted into program code using any of the high-level languages.**

- **During testing, all the modules are tested together to ensure that the overall system works well as a whole product.**

Problem Solving Strategies

- **In software deployment, training, and support phase, the software is installed or deployed in the production environment.**

- **Maintenance and enhancements are ongoing activities that are done to cope with newly discovered problems or new requirements**

Algorithm

- A formally defined procedure
- It gives the step-by-step description of how to arrive at a solution
- It provides a blueprint for writing a program to solve a particular problem

Algorithm

- In order to qualify as an algorithm, a sequence of instructions must posses the following characteristics:
  - Be precise
  - Be unambiguous
  - No instruction should be repeated infinitely
  - After an algorithm is terminated, the desired result must be obtained

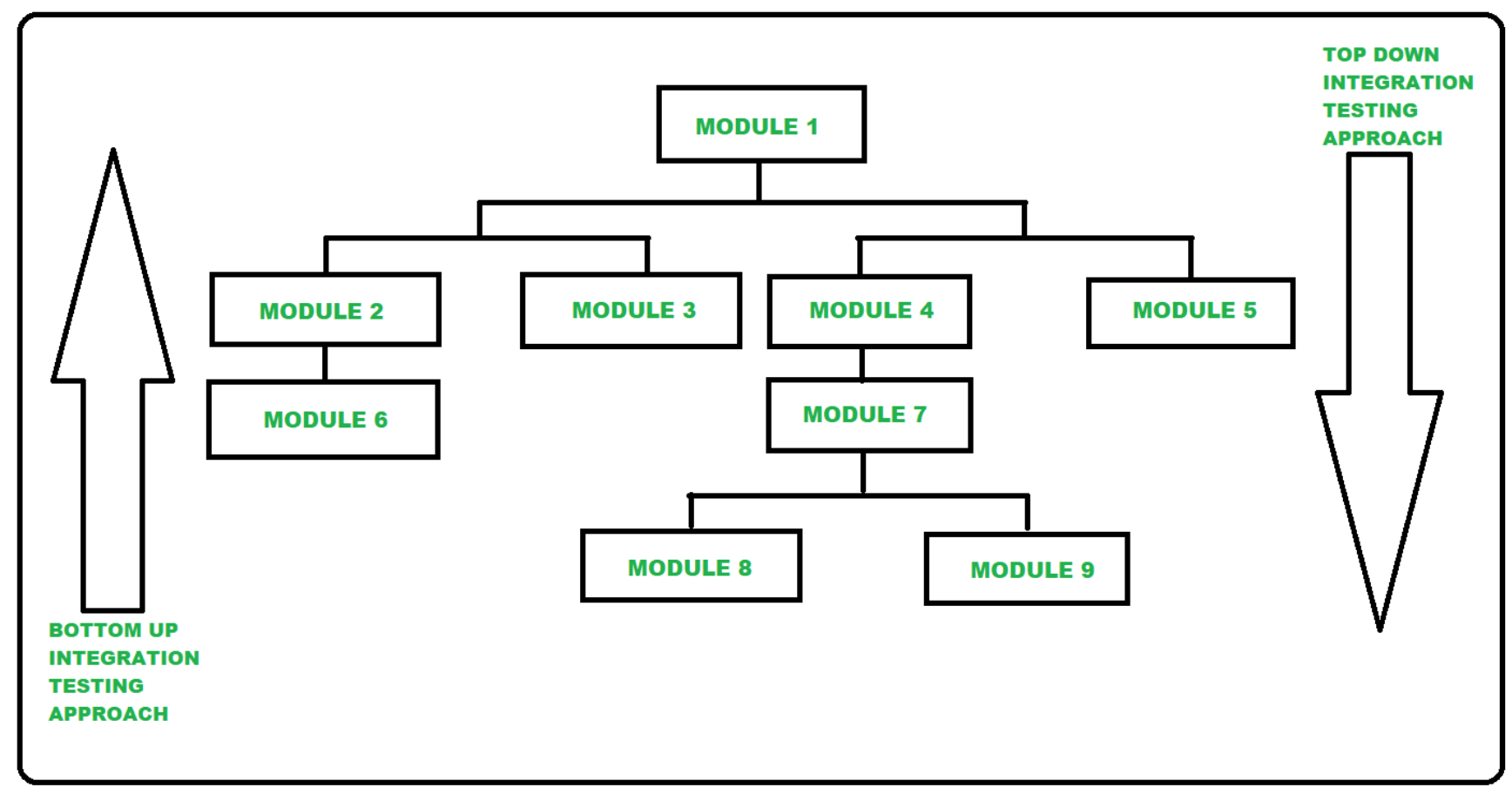
Algorithm

- **Different Approaches to Design an Algorithm**
- ***Top-down approach*** **starts by dividing the complex algorithm into one or more modules.**

Algorithm

- ***Bottom-up approach*** **is just the reverse of top-down approach.**
- **In the bottom-up design, we start with designing the most basic or concrete modules and then proceed towards designing higher level modules.**

Algorithm


MODULE 1
MODULE 2
MODULE 3
MODULE 4
MODULE 5
MODULE 6
MODULE 7
MODULE 8
MODULE 9
TOP DOWN INTEGRATION TESTING APPROACH
BOTTOM UP INTEGRATION TESTING APPROACH

Algorithm

- Whether top-down should be followed or a bottom-up is dependent on the application
- Top-down approach follows a stepwise refinement by decomposing the algorithm into manageable modules
- In bottom-up approach, several modules are grouped together to form higher level module

Control Structures in Algorithm

- **Sequence**
- **Decision**
- **Repetition**
- **Recursion**

Control Structures in Algorithm

- **Sequence** means that each step of the algorithm is executed in the specified order.
- **Decision** statements are used when the outcome of the process depends on some condition

Control Structures in Algorithm

- **Repetition, which involves executing one or more steps for a number of times, can be implemented using constructs such as the while, do-while, and for loops.**

Control Structures in Algorithm

- **Recursion is a technique of solving a problem by breaking it down into smaller and smaller sub-problems until you get to a small enough problem that it can be easily solved.**

Control Structures in Algorithm

- **Recursion involves a function calling itself until a specified condition is met.**

Control Structures: Sequence

```
Step 1 : Input first number as A
Step 2 : Input second number as B
Step 3 : Set Sum = A + B
Step 4 : Print Sum
Step 5 : End
```

Control Structures: Decision

```
Step 1 : Input first number as A
Step 2 : Input second number as B
Step 3 : IF A = B
             Print “Equal”
         ELSE
             Print “Not equal”
         [END of IF]
Step 4 : End
```

Control Structures: Repetition

```
Step 1 : [initialize] Set I = 1, N = 10
Step 2 : Repeat Steps 3 and 4 while I <= N
Step 3 : Print I
Step 4 : SET I = I + 1
         [END OF LOOP]
Step 5 : End
```

Example: Algorithm for swapping two values

Example: Algorithm for swapping two values

- Step1: Input the 1st number as A
- Step2: Input the 2nd number as B
- Step3: set temp = A
- Step4: set A = B
- Step5: set B = temp
- Step6: Print A, B
- END

Example: Algorithm to Add N natural numbers

Example: Algorithm to Add N natural numbers

- Step1: Input N
- Step2: set P = 1, sum = 0
- Step3: Repeat steps 4 and 5 while P <= N
- Step4: set sum = sum + P
- Step5: set P = P + 1

  [END OF LOOP]
- Step6: Print sum
- END

Thank You

Dr. Abdul Kayom


Asst. Prof, (SENSE)


kayomabdul@vitap.ac.in


8474860025